

ASSIGNMENT

Library Inventory Management System

Stack	MongoDB, Express.js, React.js, Node.js	Deadline	72 Hours from Assignment Date
-------	--	----------	-------------------------------

1. Overview

You are required to build a Library Inventory Management System using the MERN stack (MongoDB, Express.js, React.js, Node.js). This assignment is designed to evaluate your foundational ability to design, develop, and deliver a full-stack web application within a defined timeline.

The system should allow librarians to manage books and members through a protected interface. The focus is on clean implementation of JWT-based authentication and CRUD operations.

2. Project Objectives

By completing this assignment, you will demonstrate your ability to:

- Build a RESTful API using Node.js and Express.js
- Design and implement a MongoDB database schema using Mongoose
- Develop a frontend using React.js with basic state management
- Implement JWT-based authentication and protected routes
- Perform full CRUD operations for Books and Members
- Apply basic form validation and error handling

3. Functional Requirements

3.1 Authentication Module

- Librarian registration (create account with name, email, password)
- Librarian login — returns a JWT token on success
- JWT stored in localStorage on the frontend
- Protected routes — unauthenticated users are redirected to the login page

3.2 Book Management Module

- Add a new book with fields: Title (required), Author (required), ISBN (required, unique, 13-digit), Genre (dropdown: Fiction, Non-Fiction, Science, History, Technology, Other), Total Copies (integer, minimum 1)

- View all books in a simple table
- Edit book details
- Delete a book

3.3 Member Management Module

- Add a new member with fields: Full Name (required), Email (required, unique), Phone Number (10-digit), Membership Type (dropdown: Student, Faculty, Public)
- View all members in a simple table
- Edit and delete member records

4. Technical Requirements

4.1 Backend (Node.js + Express.js + MongoDB)

Use Express.js to build RESTful APIs and Mongoose for schema definition. Implement the following API endpoints:

Method	Endpoint	Description	Auth Required
POST	/api/auth/register	Register a new librarian	No
POST	/api/auth/login	Login, returns JWT token	No
GET	/api/books	Get all books	Yes
POST	/api/books	Add a new book	Yes
PUT	/api/books/:id	Update book details	Yes
DELETE	/api/books/:id	Delete a book	Yes
GET	/api/members	Get all members	Yes
POST	/api/members	Add a new member	Yes
PUT	/api/members/:id	Update member details	Yes
DELETE	/api/members/:id	Delete a member	Yes

- Use environment variables via .env for DB URI, JWT secret, and PORT
- Implement basic centralized error handling middleware
- Hash passwords using bcrypt before storing

4.2 Frontend (React.js)

- Use React.js with Hooks for the UI
- Use React Router DOM for navigation (Login, Books, Members pages)
- Use Axios for API calls

- Protect routes — redirect to login if no JWT token is present
- All forms must have basic client-side validation (required fields, format checks)
- Styling is flexible — plain CSS, Tailwind, or any UI library is acceptable

4.3 Database (MongoDB)

Design the following Mongoose schemas:

Collection	Key Fields	Data Types	Validations
users	name, email, passwordHash, role	String, Enum	required, unique (email), bcrypt hash
books	title, author, isbn, genre, totalCopies	String, Number, Enum	required, unique (isbn), min (copies: 1)
members	fullName, memberId, email, phone, membershipType	String, Number, Enum	required, unique (email, memberId)

5. Data Validation Rules

Fields must be validated on both the client (React) and server (Express):

Field	Type	Validation Rule
Title / Name	String	Required, min 2 characters, max 100 characters
ISBN	String (numeric)	Required, exactly 13 digits, unique in DB
Email	String	Required, valid email format, unique
Phone Number	Integer	Exactly 10 digits, numeric only
Password	String	Min 8 characters, at least 1 uppercase, 1 number, 1 special character

6. Suggested Timeline

Day	Focus
Day 1	Backend — Auth (register/login with JWT) + Books CRUD API
Day 2	Backend — Members CRUD API + Frontend Login page & routing setup
Day 3	Frontend — Books & Members pages (list, add, edit, delete) + README

7. Submission Guidelines

7.1 GitHub Repository

- Create a public GitHub repository named: library-inventory-mern
- Maintain separate folders: /client (React) and /server (Node.js)
- Make regular, meaningful commits throughout the 3 days (not one big final commit)
- Include a .gitignore file (exclude node_modules and .env files)

7.2 README.md (Mandatory)

Your README.md must include:

- Project title and description
- Tech stack used
- Features list
- Step-by-step local setup instructions (clone, install, configure .env, run)
- Environment variables required (with sample values, never real secrets)
- API documentation — list all endpoints with method, URL, request body, and response

7.3 Environment Configuration

Include a .env.example file in the /server folder with the following keys:

```
PORT=5000
MONGO_URI=mongodb+srv://<user>:<password>@cluster.mongodb.net/library
JWT_SECRET=your_secret_key_here JWT_EXPIRES_IN=7d
```

7.4 Assignment Submission via Email

- Compress your project folder (excluding node_modules and .env) and name it: library-inventory-mern_YourName.zip
- Email subject: [MERN Intern Assignment] Library IMS — YourName — Submission
- Email body: include your full name, contact number, and GitHub repository link